

Disaster Damage Assessment from Satellite Images

Project Documentation: Custom CNN, U-Net, Swin-Tiny, and SAM2 Pipeline

This document explains the complete project pipeline: dataset preparation, crop generation, custom U-Net segmentation, CustomDualInputCNN, Swin-Tiny classification, SAM2-based segmentation, evaluation, limitations, and future improvements.

Final concept:

SAM2 / U-Net -> building localization and segmentation

Custom CNN / Swin-Tiny -> building damage classification

1. Project Overview

This project performs building-level disaster damage assessment using pre-disaster and post-disaster satellite images from the xBD / xView2 dataset.

The goal is to classify each building into one of four damage categories:

```
no-damage  
minor-damage  
major-damage  
destroyed
```

The project is divided into two main tasks:

1. Building localization / segmentation
2. Building damage classification

The final working pipeline uses xBD annotations and SAM2 for building segmentation/localization, then uses pre/post building crops with Swin-Tiny or CustomDualInputCNN for damage classification.

2. Dataset: xBD / xView2

2.1 What is xBD?

xBD is a disaster damage assessment dataset based on satellite imagery. It contains pre-disaster images, post-disaster images, building polygons/annotations, and damage labels for buildings.

The dataset is useful because disaster damage assessment is not only an image classification problem. It is a change detection problem: the model needs to compare the same building before and after the disaster.

```
the building before the disaster  
vs  
the same building after the disaster
```

3. Data Preparation

3.1 Image Pairs

Each sample consists of a pre-disaster image and a post-disaster image showing the same geographic area.

3.2 Annotation Parsing

The xBD annotation files are JSON files containing building polygons. The notebook parses the annotation file and extracts the building polygon, bounding box coordinates, damage label, image ID, and building ID.

The polygon is converted into a bounding box:

```
polygon -> [x1, y1, x2, y2]
```

This bounding box is later used to crop the same building from both pre-disaster and post-disaster images.

4. Crop Generation

4.1 What is a Crop?

A crop is a small image patch extracted from the original satellite image. Instead of training on the full image, the notebook cuts out each building separately.

```
pre-disaster full image -> pre-disaster building crop
post-disaster full image -> post-disaster building crop
```

One classification sample is:

```
pre_crop + post_crop + damage_label
```

4.2 Why Use Crops?

Full satellite images contain roads, trees, fields, water, clouds, and other buildings. Damage labels are assigned to individual buildings, so the classifier should focus on the building itself.

4.3 Crop CSV Files

The crop CSV files act as an index for generated crop images. They do not store the actual pixels; they store paths and metadata.

```
pre_crop_path
post_crop_path
label
label_name
image_id
building_id
bbox coordinates
split
```

5. Balanced Subsets

The xBD dataset is imbalanced. Usually, there are far more no-damage buildings than damaged buildings.

If trained normally, the model may become biased toward predicting no-damage. Balanced subsets reduce this issue by sampling a similar number of examples from each class.

```
no-damage: 10,000
minor-damage: 10,000
major-damage: 10,000
destroyed: 10,000
```

This improves fairness across classes and makes macro F1-score more meaningful.

6. Custom U-Net Segmentation Model

6.1 Purpose of U-Net

The custom U-Net is used for building segmentation. Its task is to receive a satellite image and output a predicted building mask.

```
Input satellite image -> predicted building mask
```

A mask is a pixel-level output where building pixels are 1 and background pixels are 0. U-Net answers: Where are the buildings? It does not classify damage.

6.2 U-Net Architecture

U-Net has an encoder-decoder structure:

```
Encoder -> Bottleneck -> Decoder
```

The encoder reduces image resolution while increasing feature channels. It learns edges, textures, roof patterns, and building shapes. The bottleneck captures high-level semantic information. The decoder upsamples features back to the original image size.

Skip connections pass feature maps from the encoder to the decoder. This helps recover precise building boundaries by combining high-level understanding with precise localization.

6.3 U-Net Loss Function

The U-Net uses BCEWithLogitsLoss plus Dice Loss. BCE evaluates each pixel individually, while Dice loss measures overlap between predicted and ground-truth masks. Dice is helpful because most pixels are background and building pixels are fewer.

6.4 U-Net Metrics

Segmentation can be evaluated using Dice score, IoU, pixel accuracy, precision, and recall. Dice and IoU are usually the most important because they measure mask overlap.

7. Custom CNN Damage Classification Model

7.1 Model Name

The custom CNN architecture is called CustomDualInputCNN. A presentation-friendly name is Custom Dual-Input CNN for Before-and-After Damage Classification.

7.2 Why Siamese-Style?

The model is Siamese-style because it processes two related inputs using a shared feature extractor:

```
pre-disaster crop -> shared CNN branch -> pre features
post-disaster crop -> shared CNN branch -> post features
```

Both inputs represent the same building at different times.

7.3 Custom CNN Architecture

The shared CNN feature extractor contains convolution blocks with Conv2D, BatchNorm2D, ReLU, MaxPool2D, and Dropout2D. The channel depth increases through the network.

```
3 -> 32 -> 64 -> 128 -> 256
```

Adaptive average pooling converts each crop into a feature vector.

7.4 Feature Fusion

After extracting features, the model fuses pre features, post features, and their absolute difference:

```
[pre_features, post_features, |pre_features - post_features|]
```

This is important because the model needs to learn what changed between the two images.

7.5 Output

The final classifier outputs four logits: no-damage, minor-damage, major-damage, and destroyed. Softmax converts these logits into probabilities, and the class with the highest probability becomes the prediction.

7.6 Why This Architecture Makes Sense

Damage assessment is a comparison task. A model using only the post-disaster crop may confuse old roof texture, shadows, vegetation, or unusual shapes with damage. The dual-input design compares the same building before and after the disaster.

8. Swin-Tiny Damage Classification Model

8.1 Purpose

Swin-Tiny is the stronger pretrained damage classification model. It also uses a dual-input design: pre crop to Swin backbone, post crop to the same Swin backbone, then feature fusion.

8.2 Why Swin-Tiny?

Swin-Tiny uses shifted-window attention. Instead of applying attention to the entire image at once, it divides the image into local windows and applies attention inside each window. In later layers, the windows are shifted so information can move between neighboring regions.

```
image crop
-> patches/windows
-> attention inside each window
-> shifted windows
-> cross-window information sharing
```

This helps capture both local details and surrounding context, which is useful for roof damage, debris, shape changes, collapsed structures, and surrounding disaster effects.

8.3 Why Not Only ResNet?

ResNet is a strong CNN baseline, but it mainly relies on convolutional local features. Swin-Tiny can capture both local and contextual relationships using shifted-window attention.

8.4 Why Swin-Tiny Instead of Larger Swin Models?

Swin-Tiny gives a good balance between performance, training time, and GPU memory. It is more practical for an RTX 4060 Ti than much larger transformer models.

9. SAM2 Segmentation Pipeline

9.1 Purpose of SAM2

SAM2 is used for building segmentation/localization. It does not classify damage. SAM2 answers: Where is the building? The Swin/CNN classifier answers: How damaged is the building?

9.2 Why Use SAM2?

SAM2 is a prompt-based segmentation model. It can segment objects when given prompts such as points, boxes, or masks. In this project, xBD building boxes are used as SAM2 prompts.

9.3 SAM2 Setup

```
SAM2_CONFIG = "configs/sam2.1/sam2.1_hiera_b+.yaml"
SAM2_CHECKPOINT = "D:/Deeplearning project/sam2/sam2.1_hiera_base_plus.pt"
```

The notebook confirms that SAM2 is available, the checkpoint exists, and CUDA runs on the RTX 4060 Ti.

9.4 Single-Building Inference

Single-building mode uses one prompt: one box or point produces one SAM2 mask, one crop pair, and one prediction.

9.5 All-Buildings Inference

The preferred pipeline loops through all xBD building annotations. For each building, it converts the polygon to a bounding box, uses the box as a SAM2 prompt, refines the mask, crops the same region from pre and post images, and predicts damage using the classifier.

```
xBD polygon -> bounding box -> SAM2 box prompt -> SAM2 refined mask
-> refined/expanded bbox -> pre crop + post crop -> Swin classifier -> damage prediction
```

9.6 SAM2 Output

The final visual output draws colored boxes over buildings, such as green for no-damage, yellow/orange for minor damage, orange for major damage, and red for destroyed.

10. Full End-to-End Pipeline

- 1. Load pre-disaster image and post-disaster image
- 2. Read xBD annotation JSON
- 3. Extract building polygons
- 4. Convert polygons to bounding boxes
- 5. Use each bounding box as a SAM2 prompt
- 6. SAM2 generates/refines building masks
- 7. Convert masks to refined boxes
- 8. Crop the same building region from pre and post images
- 9. Resize and normalize crops
- 10. Send crop pair to Swin-Tiny or CustomDualInputCNN
- 11. Predict damage class
- 12. Draw prediction on post-disaster image
- 13. Save CSV, figures, reports, and confusion matrix

11. Evaluation

11.1 Classification Metrics

The project evaluates damage classification using accuracy, precision, recall, F1-score, macro F1, weighted F1, and confusion matrix.

11.2 Why Macro F1 is Important

The dataset is imbalanced. Accuracy alone can be misleading because a model can predict no-damage too often and still appear good. Macro F1 treats each class equally.

```
F1(no-damage)
F1(minor-damage)
F1(major-damage)
F1(destroyed)
then average them equally
```

11.3 Confusion Matrix

A confusion matrix compares ground-truth labels versus predicted labels. Rows are the true classes and columns are the predicted classes. The diagonal contains correct predictions.

11.4 Single-Image Evaluation

For one image, the support may be very small. Such results are useful as a demo but are not enough to judge the whole model. Full evaluation should use many buildings/images.

12. Model Roles Summary

Component	Role	Input	Output
U-Net	Building segmentation baseline	Satellite image	Building mask
SAM2	Prompt-based segmentation/refinement	Pre image + box prompt	Building mask
CustomDualInputCNN	Damage classification baseline	Pre crop + post crop	Damage class
Swin-Tiny	Stronger damage classifier	Pre crop + post crop	Damage class

Simple explanation:

U-Net / SAM2 = where is the building?
Custom CNN / Swin = how damaged is the building?

13. Why Use SAM2 + Swin-Tiny?

The SAM2 + Swin-Tiny pipeline separates the problem into two logical parts. SAM2 focuses on accurate building segmentation, while Swin-Tiny focuses on before/after damage comparison.

This avoids forcing one model to do everything. SAM2 isolates the building region, and Swin-Tiny compares the pre/post crops.

14. Limitations

- SAM2 requires prompts such as boxes or points. In this notebook, xBD annotations provide those prompts.
- For real-world deployment, an automatic building detector or segmentation model would be needed to generate prompts automatically.
- The dataset is imbalanced, so models may become biased toward no-damage.
- Some damage categories are visually subtle, such as minor-damage versus no-damage or major-damage versus destroyed.
- The method assumes pre and post images are aligned well enough so the same coordinates correspond to the same building.

15. Future Improvements

- Add an automatic building detector to replace xBD box prompts.
- Fine-tune SAM2 on building masks for satellite imagery.
- Train on a larger balanced dataset.
- Try stronger pretrained backbones such as ConvNeXt or larger Swin models.
- Add test-time augmentation.
- Improve handling of clouds, shadows, and low-quality crops.
- Evaluate across more disaster types and geographic regions.
- Add a web interface for uploading pre/post image pairs.

16. Presentation Summary

This project performs building-level disaster damage assessment using pre- and post-disaster satellite images. First, xBD annotations provide building locations. These locations are converted into bounding boxes and used as prompts for SAM2, which refines the building segmentation.

Then the same building region is cropped from the pre-disaster and post-disaster images. These two crops are passed into a trained dual-input classifier. The custom CNN is a Siamese-style baseline, while Swin-Tiny is the stronger pretrained model.

The classifier compares the before-and-after building features and predicts one of four damage classes: no-damage, minor-damage, major-damage, or destroyed. Finally, the notebook saves annotated prediction images, CSV files, classification reports, and confusion matrices.

17. Key Point to Remember

SAM2 does not classify damage.
SAM2 segments/localizes buildings.

Swin-Tiny and CustomDualInputCNN do not find buildings by themselves.
They classify damage from pre/post building crops.

So the final system is:

SAM2 -> building segmentation
Swin/CNN -> damage classification